

Computación Concurrente. Práctica 1.

Martínez Mejía Eduardo
Victoria Morales Ricardo Maximiliano
Torres Nava Hazel

27 de Agosto de 2026

1. <https://www.evanjones.ca/software/threading-linus-msg.html>.

Comparte en máximo 4 líneas de computadora a qué se refiere Linus Torvalds con un contexto de ejecución y cómo se relaciona con la definición en la sección 1 de esta práctica.

Es un conjunto de estados de la computadora asociados a una tarea. Verlo así permite dejar de ver sólo hilos y procesos como varios caminos bifurcados actuando sobre un mismo plan, sino ver módulos con datos propios que pueden manejar y compartir según se requiera, sin la limitación de depender completamente de un padre, teniendo total libertad de ejecución.

2. ¿Cuántos hilos tiene disponibles tu computadora?

Ejecuta `Runtime.getRuntime().availableProcessors()`, si son más de uno en el equipo escriban el de cada uno.

- **Eduardo:**

```
PS C:\Users\blood\Documents\ESCUELA\Ciencias Compu\C. Concurrente> java .\NumeroProcesadores.java
Número de procesadores: 20
```

- **Ricardo:**

```
ricky@ThinkPad-T14-Gen-21:~/Universidad/Semestres/9noSemestre/ComputacionConcurrente/Practicas/Practica1$ cat hilos.java
public class Uso{

    public static void main(String[] args){
        System.out.print("Numero de hilos de mi computadora: ");
        System.out.println(Runtime.getRuntime().availableProcessors());
    }
}

ricky@ThinkPad-T14-Gen-21:~/Universidad/Semestres/9noSemestre/ComputacionConcurrente/Practicas/Practica1$ java hilos.java
Numero de hilos de mi computadora: 8
```

- **Hazel:**

```
Num. de procesadores: 8
```

3. Revisa el programa *Determinante* concurrente y responde ¿Cuánto tiempo tarda en ejecutarse?.

- Eduardo:

```
PS C:\Users\blood\Documents\ESCUELA\Ciencias Compu\C. Concurrente> java .\ProgramasPractica1\practica1\DeterminanteConcurrente.java
Program took 890400ns, result: -18
```

- Ricardo:

```
ricky@ThinkPad-T14-Gen-21: ~/Universidad/Semestres/9noSemestre/ComputacionConcurrente/Practicas/Practica1/ProgramasPractica1/practica1$ \
> java DeterminanteConcurrente.java
Program took 845819ns, result: -18
```

- Hazel:

```
Program took 961189ns, result: -18
```

4. El programa *Determinante* concurrente está implementado extendiendo la clase *Thread*. Implementa el programa utilizando la interfaz *Runnable*.

Partiendo del programa *Determinante*, cambiamos la definición de la clase para que implemente *Runnable*, cambiando su nombre a *DeterminanteConcurrenteRunnable* para distinguirla.

Como cada hilo que necesitamos para dividir cada conjunto de multiplicaciones a efectuar al aplicar la regla de *Sarrus*, debemos crear los objetos runnable necesarios, en este caso son 6 al ser una matriz 3×3 , con lo que le podemos pasar a cada uno los valores de la matriz con los que debe operar.

De aquí partimos para crear un thread para cada runnable y comenzar la ejecución, siguiendo exactamente las mismas instrucciones que cuando se decidió extender *Thread*.

5. Implementa el programa *Determinante* concurrente de forma secuencial.

```
public class DeterminanteSecuencial {
    static int determinante;
    static int n_prueba = 3;
    static int matriz_prueba[][] = { { 1, 2, 2 }, { 1, 0, -2 }, { 3, -1, 1 } };

    public static int determinanteMatriz3x3(int matriz[][], int n_prueba) {
        // Calculamos las diagonales principales
        int part1 = matriz[0][0] * matriz[1][1] * matriz[2][2];
        int part2 = matriz[1][0] * matriz[2][1] * matriz[0][2];
        int part3 = matriz[2][0] * matriz[0][1] * matriz[1][2];

        // Calculamos las diagonales inversas
        int part4 = matriz[2][0] * matriz[1][1] * matriz[0][2];
        int part5 = matriz[1][0] * matriz[0][1] * matriz[2][2];
        int part6 = matriz[0][0] * matriz[2][1] * matriz[1][2];

        // Sumamos y restamos
        return part1 + part2 + part3 - part4 - part5 - part6;
    }

    public static void main(String[] args) {
        long startTime = System.nanoTime();
        determinante = determinanteMatriz3x3(matriz_prueba, n_prueba);
        long endTime = System.nanoTime();

        System.out.println("Program took " +
            (endTime - startTime) + "ns, result: " + determinante);
    }
}
```

1. Para calcular el determinante de una matriz 3x3, primero calculamos el producto de las tres diagonales principales
2. posteriormente, calculamos las diagonales secundarias
3. Tomamos los primeros tres resultados y los sumamos
4. Y por ultimo, a esa suma le restamos los ultimos tres resultados

6. Implementa del programa *Determinante* concurrente para dos hilos (en vez de seis).

«««< HEAD

===== >>>> 67e3870 (update: pdf) El código fuente es el del archivo *DeterminanteConcurrenteDosHilos.java*.

1. Nos basamos en el archivo que tenía 6 hilos.
2. Dividimos el trabajo de cálculo en 2 hilos, ahora cada hilo iba a trabajar con 9 operandos. Dividimos la resta final del cálculo del determinante en dos pasos. Cada hilo ahora sólo multiplica sus 9 parámetros.
3. Finalmente se resta el producto de cada hilo para obtener el determinante.

7. Compara las 3 implementaciones: el programa *Determinante* concurrente para dos hilos, para seis hilos y el programa secuencial.

Responde: ¿A qué se debe el orden en el que se ordenan los tiempos de ejecución de cada programa?

Los tiempos de ejecución de las diferentes versiones del programa resultaron ser, de mayor a menor las siguientes:

- Secuencial
- Concurrente de 2 hilos
- Concurrente de 6 hilos

```
PS C:\Users\blood\Documents\ESCUELA\Ciencias Compu\C. Concurrente\Práctica 1\concurrency-masters\src>
java .\DeterminanteConcurrente.java
Program took 880300ns, result: -18
PS C:\Users\blood\Documents\ESCUELA\Ciencias Compu\C. Concurrente\Práctica 1\concurrency-masters\src>
java .\DeterminanteConcurrenteDosHilos.java
Program took 494500ns, result: -18
PS C:\Users\blood\Documents\ESCUELA\Ciencias Compu\C. Concurrente\Práctica 1\concurrency-masters\src>
java .\DeterminanteSecuencial.java
Program took 3100ns, result: -18
```

Analizando el algoritmo base nos podemos dar cuenta de que realmente el calcular la determinante de una matriz 3×3 requiere de pocas operaciones, por lo que podemos pensar que el aumento de tiempo de ejecución que observamos se debe al costo de la creación e inicialización de cada hilo, lo que agrega tiempo extra, por lo que mientras más hilos, más tiempo tomamos. Podemos entonces ver que en este problema específico no ganamos velocidad real en la práctica.

8. Si utilizas la Ley de Amdahl entre el programa *Determinante* concurrente para dos hilos y el programa secuencial. ¿El resultado es mayor o menor a 1? ¿Por qué?

Al ejecutar el programa secuencial, éste tardó $a = 3501ns$, y al ejecutar el de dos hilos tardó $b = 896995ns$. Es decir, a/b es menor a 1. Por lo tanto, no hay aceleración con dos hilos que con secuencial. Esto probablemente se debe a que estos cálculos son muy simples y rápidos para un procesador normal, y tarda menos que todo el proceso de crear hilos.

9. Describe con tus propias palabras en máximo dos líneas para qué sirve el método *join()*. Si no utilizas el método *join()* en *Determinante Concurrente*, ¿sigue funcionando?

Detiene al hilo principal para que los hilos a los que llama terminen su trabajo antes de continuar, siendo útil para juntar sus resultados.

Si se comentan todos los *join()* deja de funcionar y el resultado arroja 0, pues cada hilo termina después del principal *determinanteMatriz3x3*, y al instante se suman los valores *partial* de cada hilo, que se inicializaron en 0.